



Date: __ / __ / __

Experiment 9

9. a. You're tasked with implementing Dijkstra's algorithm in the C programming language to find the Single Source Shortest Path for a given weighted graph. The algorithm should efficiently determine shortest path from one specific vertex to all other vertices in a graph, optimizing the total distance traveled.

For more information, students are requested to read chapter 24.3 from the textbook 2. (Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms. MIT press.)

Type of Lab Activity: Exercise

Aim	Material	Methods	Answer
Given	Given	Given as Reading Chapters	Open

Program

(Only for faculty reference, should not be shared to students)

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define V 9

int minDistance(int dist[], int sptSet[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++)
        if (sptSet[v] == 0 && dist[v] <= min)
            min = dist[v], min_index = v;

    return min_index;
}

void printSolution(int dist[], int n) {
    printf("Vertex \t Distance from Source\n");
    for (int i = 0; i < V; i++)
        printf("%d \t %d\n", i, dist[i]);
}
```



```
void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int sptSet[V];

    for (int i = 0; i < V; i++)
        dist[i] = INT_MAX, sptSet[i] = 0;

    dist[src] = 0;

    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, sptSet);
        sptSet[u] = 1;

        for (int v = 0; v < V; v++)
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v] <
                dist[v])
                dist[v] = dist[u] + graph[u][v];
    }

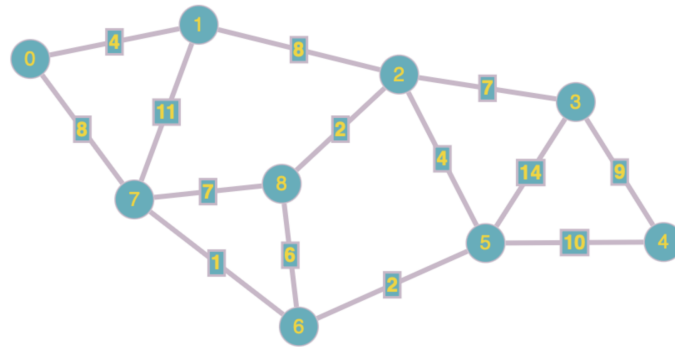
    printSolution(dist, V);
}

int main() {
    int graph[V][V] = { {0, 4, 0, 0, 0, 0, 0, 8, 0},
                        {4, 0, 8, 0, 0, 0, 0, 11, 0},
                        {0, 8, 0, 7, 0, 4, 0, 0, 2},
                        {0, 0, 7, 0, 9, 14, 0, 0, 0},
                        {0, 0, 0, 9, 0, 10, 0, 0, 0},
                        {0, 0, 4, 14, 10, 0, 2, 0, 0},
                        {0, 0, 0, 0, 0, 2, 0, 1, 6},
                        {8, 11, 0, 0, 0, 0, 1, 0, 7},
                        {0, 0, 2, 0, 0, 0, 6, 7, 0} };

    dijkstra(graph, 0);
    return 0;
}
```



Sample Output



Vertex	Distance from Source
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14